

MODULE 6 — Memory (Page) Replacement

Senior SWE Interview Track — Operating Systems

6.0 Why Replacement Exists (framing)

When RAM is full and a page must come in, the kernel evicts a victim. The same math governs **every cache you will ever build** — CPU caches, page cache, Redis `maxmemory-policy`, CDN eviction — which is why interviewers love this module: it's OS theory you use weekly. Goal: minimize fault/miss rate; the unbeatable oracle is **OPT/Belady's algorithm** (evict the page used farthest in the future) — unimplementable, but the benchmark. Clean pages are cheap to evict (drop); dirty pages must be written back first (the dirty bit from Module 5 pays off here).

6.1 FIFO

1. Why Interviewers Ask This

Baseline + the setup for **Belady's anomaly**. Also a hand-simulation favorite.

2. Core Concept

Evict the page that has been *resident longest*, regardless of use. Queue: evict head, insert at tail.

3. Internal Working

Ring buffer/queue of frames; $O(1)$ eviction; no per-access bookkeeping (nothing updates on a hit — that's both its cheapness and its flaw).

4. ASCII Diagram

```
Ref string: 1 2 3 4 1 2 5 1 2 3 4 5      3 frames
[1][ ][ ] F  [1][2][ ][ ] F  [1][2][3] F  [4][2][3] F  (evict 1)
[4][1][3] F (evict 2) ... total = 9 faults
Flaw: evicts "oldest" even if it's the hottest page.
```

5. Real Production Example

Simple firmware/embedded caches; message-buffer recycling; anywhere metadata budget is ~zero. Rarely acceptable for real page caches.

6. Advantages

Trivial, $O(1)$, no hit-path overhead, no scan cost.

7. Trade-offs

Ignores recency/frequency entirely → evicts hot pages; suffers **Belady's anomaly** (more frames can mean *more* faults — 6.5).

8. Common Mistakes

- Updating the queue on hits (that turns it into LRU — in FIFO, hits do nothing).
- Sloppy hand simulation: mark hit/fault per step, track queue order explicitly.

9. Performance Implications

Typically 20–50% worse fault rate than LRU on workloads with locality; fine when accesses are uniform/scan-like (no locality to exploit).

10–11. Interview & Follow-ups

- “Simulate this reference string with 3 frames.” “What’s FIFO’s core flaw?” “Which algorithms are anomaly-free?” (stack algorithms: LRU, OPT)

12. Coding/Debugging Scenario

Implement FIFO cache in 10 lines (deque + set); use as the control in an eviction A/B benchmark.

13. Best Practices

Use FIFO only when hit-path cost must be zero or access patterns lack locality; otherwise start at LRU/Clock.

14. Practice Questions

1. Count faults: `7 0 1 2 0 3 0 4 2 3 0 3 2` with 3 frames (answer: 9... verify by hand).
2. Construct a workload where FIFO beats LRU. (looping scan slightly larger than cache — LRU thrashes at 0% hit, FIFO too; sequential one-shot scans — tie; adversarial LRU patterns)

6.2 LRU (Least Recently Used)

1. Why Interviewers Ask This

Double-duty: OS theory **and** the #1 data-structure design question (“Design an LRU cache” — LeetCode 146, asked constantly at Meta/Amazon). Also probes why true LRU is infeasible in hardware.

2. Core Concept

Evict the page unused for the longest time — exploits **temporal locality** (recent past predicts near future). It’s OPT’s mirror: OPT looks forward, LRU looks backward.

3. Internal Working

- **Software (the interview classic)**: hash map (key → node) + doubly-linked list in recency order. Hit: move node to front, $O(1)$. Evict: remove tail, $O(1)$.
- **Why the OS can’t do true LRU**: it would need to timestamp/reorder on *every memory access* — hardware only gives a per-page **accessed bit**. So kernels approximate (Clock, 6.4; Linux uses active/inactive lists, and MGLRU in newer kernels).
- LRU is a **stack algorithm** (contents with k frames $\subseteq$ contents with $k+1$) → immune to Belady’s anomaly.

4. ASCII Diagram

```
HashMap: k -> node          Doubly-linked list (MRU ... LRU)
get(k):  lookup node, unlink, insert at head      O(1)
put(k,v): if full: evict tail (also remove from map); insert head

head [k4] <-> [k9] <-> [k2] <-> [k7] tail  <- evict k7
Same ref string as FIFO example: LRU = 10 faults there; usually LRU wins
on real (locality-heavy) traces, not always on adversarial ones.
```

5. Real Production Example

- Redis `maxmemory-policy allkeys-lru` (actually *approximate*: samples 5 random keys, evicts oldest — near-LRU at a fraction of the cost; a great “theory vs practice” talking point).
- MySQL InnoDB buffer pool: LRU with a **midpoint insertion** (new pages enter 3/8 from the tail) so table scans can’t flush the hot set — “scan resistance”.
- CDN/memcached eviction; CPU caches use pseudo-LRU bits.

6. Advantages

Best simple predictor under temporal locality; $O(1)$ software implementation; anomaly-free; intuitive to reason about.

7. Trade-offs

Per-access bookkeeping (list ops + locking → contention in concurrent caches); zero **frequency** awareness (one touch of a cold page makes it “hot”); catastrophically bad on sequential scans slightly larger than the cache (loop of $N+1$ pages over N frames → 100% miss).

8. Common Mistakes

- LRU-cache coding slips: forgetting to remove evicted key from the map; not moving to front on **put** of existing key; $O(n)$ list search instead of storing node refs in the map.
- Claiming OSes implement true LRU.
- Not knowing the scan-pollution weakness (leads into LFU/2Q/midpoint discussions).

9. Performance Implications

Software hit path $\sim O(1)$ but a shared lock on the list serializes; sharded/segmented LRU or per-CPU structures fix it. Fault-rate: excellent with locality; degenerate on loops/scans (mention MRU as the fix for pure loops).

10. Common Interview Questions

- “Design LRU cache with $O(1)$ get/put.” (write it cold)
- “Why can’t the kernel use true LRU?” “How does Redis approximate LRU?”

11. Follow-up Questions

- “Make your LRU thread-safe — now make it scale.” (lock striping, per-shard LRU)
- “How does InnoDB stop a full-table scan from evicting the working set?” (midpoint LRU)
- “LRU-K / 2Q / ARC — what do they add?” (frequency/scan resistance)

12. Coding/Debugging Scenarios

- Cache hit rate collapses every night at 02:00 → batch job scans a huge table through the same cache → pollution; fix: scan-resistant policy, separate cache, or bypass hints (**MADV_DONTNEED**-style).

13. Best Practices

Segment or midpoint-insert for scan resistance; sample-based approximation at large scale; measure hit ratio *per class of traffic*.

14. Practice Questions

1. Implement LRU cache (map + DLL) in your language, then add TTL.
2. Simulate loop 1..4 repeatedly with 3 frames under LRU vs MRU — fault rates?
3. Sketch sharded LRU for 10M entries, 32 threads.

6.3 LFU (Least Frequently Used)

1. Why Interviewers Ask This

The frequency-vs-recency trade-off, plus a harder design variant (“Design LFU cache”, LeetCode 460) that filters senior candidates.

2. Core Concept

Evict the page with the *fewest* accesses. Captures long-term popularity that LRU misses (a page touched 1000 times shouldn’t die because of a 10-second quiet spell).

3. Internal Working

- Naive: counter per page; evict min — $O(n)$ scan or heap $O(\log n)$.

- O(1) design: hash key → node; nodes grouped in **frequency buckets** (DLL per frequency, frequencies in a list); on access, move node to freq+1 bucket; evict from the lowest non-empty bucket (LRU within it for ties).
- **Aging is mandatory**: raw counters never forget — yesterday's viral object blocks today's. Fixes: periodic halving of counters, or windowed/decayed counts. Redis LFU: 8-bit *probabilistic* counter (increments with probability decreasing as count grows) + configurable decay time (**lfu-decay-time**).
- Modern practice: **TinyLFU/W-TinyLFU** (Caffeine's policy) — a tiny count-min sketch as an *admission filter* in front of a windowed LRU: only admit a new key if it's likely more popular than the eviction victim.

4. ASCII Diagram

```
freq buckets: [1: c->a] [2: d] [5: b]
access(a): move a from bucket1 to bucket2
evict: head of lowest bucket -> c
Without decay: old celebrity item at freq=50000 is immortal
Decay: every T, counter /= 2 -> popularity has a half-life
```

5. Real Production Example

- Redis **allkeys-lfu** (recommended over LRU for skewed key popularity).
- Caffeine (W-TinyLFU) — default high-performance Java cache, near-optimal hit rates in benchmarks; used across ad-tech/serving layers.
- CDNs: admission-controlled frequency caches so one-hit-wonders (~60–70% of CDN objects!) never displace hot content.

6. Advantages

Superior for skewed, stable popularity (Zipf traffic — i.e., most internet workloads); scan/one-hit-wonder resistant (new items start at freq 1).

7. Trade-offs

Cache pollution by *stale* frequency without decay; new hot items ramp slowly (cold-start penalty); heavier bookkeeping; the pure-LFU O(1) structure is fiddly.

8. Common Mistakes

- Proposing LFU without decay/aging — interviewers pounce on the “old viral video” problem.
- Ignoring tie-breaking (use LRU within a frequency bucket).
- Not knowing the sketch-based modern form (TinyLFU) — the senior-level upgrade.

9. Performance Implications

On Zipfian traces LFU/TinyLFU beat LRU hit rates by several points (huge at CDN scale: +1% hit rate = big origin-traffic cut); on shifting working sets, undecayed LFU is the *worst* of the classic policies.

10–11. Interview & Follow-ups

- “LRU vs LFU — when does each win?” “Design an O(1) LFU.” “How does Redis implement LFU in 8 bits?” “What is TinyLFU admission?”

12. Coding/Debugging Scenario

CDN node keeps evicting current-hour hot objects while month-old assets sit cached → frequency staleness; add decay or switch to W-TinyLFU-style admission + windowed LRU.

13. Best Practices

Always pair frequency with decay; use admission filters for one-hit-wonder-heavy traffic; prefer proven libraries (Caffeine) over hand-rolled LFU.

14. Practice Questions

1. Implement O(1) LFU (get/put) with frequency buckets.

- Given Zipf(1.0) traffic and a cache 1% of the corpus, argue which policy wins and why.
- Design decay: half-life 1 hour, 8-bit counters — sketch the update rule.

6.4 Clock Algorithm (Second Chance)

1. Why Interviewers Ask This

“How does the OS *actually* approximate LRU?” — Clock is the canonical answer, and it tests whether you know what the hardware accessed-bit gives you.

2. Core Concept

Frames arranged in a circle with a hand. Each page has a hardware-set **reference (accessed) bit**. To evict: advance the hand; if ref bit = 1, clear it and move on (**second chance**); if 0, evict. Recently used pages survive one sweep; untouched pages don't.

3. Internal Working

- Hardware sets the accessed bit on any read/write to the page (in the PTE); the kernel clears it — no per-access software cost (the whole point).
- Enhanced (NRU-style) variant uses (ref, dirty) pairs: prefer evicting (0,0) > (0,1) > (1,0) > (1,1) — clean-and-cold first, since dirty pages need writeback.
- Real kernels elaborate: Linux classic active/inactive lists = a two-tier clock (accessed pages promote; eviction from inactive tail; refault tracking to size the tiers); MGLRU (recent kernels) generalizes to multiple generations. FreeBSD/Mach used explicit two-handed clocks (front hand clears, back hand evicts; hand spread tunes the “window”).

4. ASCII Diagram

```

    [P3 r=1]
[P2 r=0]   [P4 r=0]
  HAND--> ^
[P1 r=1]   [P5 r=1]
    [P0 r=0]
Evict pass: P4 r=0 -> EVICT. If it were r=1: clear, advance.
Worst case: all bits 1 -> full lap (clears all) -> degrades to FIFO.
```

5. Real Production Example

Linux page-cache/anon reclaim (kswapd walks LRU lists driven by accessed bits — clock in spirit); PostgreSQL's buffer pool uses **clock-sweep** with a usage counter (0–5) — a multi-bit clock; hypervisors sample accessed bits for working-set estimation.

6. Advantages

Near-LRU hit rates at ~zero hit-path cost; O(1) amortized eviction; hardware does the tracking; no list reordering (no lock contention on access).

7. Trade-offs

Coarse (1 bit ≈ “touched since last sweep?”); sweep cost under pressure; degenerates to FIFO when everything's referenced; eviction quality depends on sweep frequency vs access rate.

8. Common Mistakes

- Not knowing the accessed bit is set by *hardware* (candidates invent per-access software hooks — that's exactly what Clock avoids).
- Forgetting the dirty-bit interaction (evicting dirty pages costs a writeback → prefer clean victims, hence enhanced clock / background flusher).

9. Performance Implications

Approximates LRU within a few percent on typical workloads at ~0 overhead — the textbook engineering trade. Under extreme memory pressure the hand spins fast (visible as `kswapd CPU` + `pgscan/pgsteal` rates) — a real triage signal.

10–11. Interview & Follow-ups

- “Explain second chance.” “What does Linux actually do?” (active/inactive or MGLRU — say “clock-family approximation of LRU with accessed bits”)
- “What happens when every page has `ref=1`?” “How does the dirty bit change victim choice?”

12. Coding/Debugging Scenario

Host under memory pressure: `sar -B` shows huge `pgscan/s` vs low `pgsteal` → reclaim scanning hard for few evictable pages (all hot/dirty) → thrashing verge; add RAM, cut working set, or tune dirty writeback.

13. Best Practices

Cite Clock when asked to build a cheap LRU-ish cache with no per-hit locking (a ring + reference flags is a legit design); watch `pgscan/pgsteal` ratios in prod.

14. Practice Questions

1. Simulate clock on ref string `1 2 3 4 1 5` with 4 frames, bits shown per step.
2. Design a lock-free-ish cache eviction using per-entry “touched” flags + a sweeper thread — what did you just reinvent?

6.5 Belady’s Anomaly

1. Why Interviewers Ask This

A counterintuitive gem: **more memory can cause more page faults**. Tests rigor (can you demonstrate it?) and theory depth (why LRU is immune).

2. Core Concept

Under FIFO, increasing frame count can *increase* faults for some reference strings. Canonical string: `1 2 3 4 1 2 5 1 2 3 4 5` → **9 faults with 3 frames, 10 with 4**.

3. Internal Working

Why: FIFO’s eviction depends on *arrival order*, and the set of resident pages with k frames is **not necessarily a subset** of the set with $k+1$ frames — the caches “diverge”. **Stack algorithms** (LRU, OPT, LFU-by-true-frequency) maintain the inclusion property: $\text{resident}(k) \subseteq \text{resident}(k+1)$ at every step → more frames can never hurt → anomaly-impossible.

4. ASCII Diagram

```
Ref: 1 2 3 4 1 2 5 1 2 3 4 5
3 frames (FIFO): F F F F F F H H F F H = 9 faults
4 frames (FIFO): F F F F H H F F F F F = 10 faults (!)
Inclusion property (LRU): 3-frame content is always a subset of
4-frame content -> extra frame can only convert misses to hits.
```

5. Real Production Example

The general lesson bites for real: any FIFO-flavored cache (some hardware TLBs/queues, naive eviction in app caches) can *regress after a capacity upgrade*. “We doubled the cache and hit rate dropped” is a real incident class — always benchmark policy × size, don’t assume monotonicity.

6–7. Advantages / Trade-offs

(A property, not a tool.) Its value: proves eviction-policy choice matters *structurally*, not just by constants; motivates stack algorithms.

8. Common Mistakes

- Claiming the anomaly applies to LRU (it cannot — inclusion property; be ready to state why).
- Being unable to reproduce the canonical example under time pressure (practice the 12-step table until mechanical).
- Overstating frequency: it's rare and workload-specific — a curiosity with a deep lesson, not a daily hazard.

9. Performance Implications

Direct: capacity planning for FIFO-like caches must be empirical. Indirect: prefer policies with the inclusion property when you want “more RAM never hurts” guarantees (miss-ratio curves monotone → simple autoscaling).

10–11. Interview & Follow-ups

- “What is Belady’s anomaly? Demonstrate it.” “Why is LRU immune?” (stack/inclusion property) “What’s Belady’s *algorithm*?” (OPT — different thing, same Belady; evict farthest-future-use; the optimality benchmark)

12. Coding/Debugging Scenario

Write a 30-line simulator (policy × frames × ref string); verify 9 → 10 on the canonical string; use the same harness to pick eviction policies with production traces — the actually-useful takeaway.

13. Best Practices

Validate cache sizing with trace-driven simulation (miss-ratio curves); don’t extrapolate “bigger is better” for non-stack policies.

14. Practice Questions

1. Hand-run the canonical string at 3 and 4 frames; produce both fault counts.
2. Prove (informally) that LRU satisfies the inclusion property.
3. Distinguish: Belady’s anomaly vs Belady’s optimal algorithm.

6.6 Thrashing (bonus — the production failure mode)

The reason this module matters operationally: when the **working set** exceeds RAM, every fault evicts a page that’s needed again soon → the system spends its time paging, not computing. Signature: CPU% low, load high, disk 100%, majflt/s high, everything crawls. Fixes: add RAM, shrink working sets (better locality, smaller heaps), cap concurrency (fewer simultaneous working sets — admission control), kill the offender (OOM killer as last resort), never “fix” by adding more swap (extends the agony). Interview line: “*Thrashing is when the aggregate working set exceeds physical memory; the cure is reducing demand or adding supply — never more swap.*”

Module 6 Cheat Sheet (one page)

Policy	Evicts	Cost	Strength	Fatal flaw
OPT (Belady)	Farthest future use	Needs oracle	Optimal benchmark	Unimplementable
FIFO	Oldest resident	$O(1)$, no hit cost	Simplicity	Ignores use; Belady’s anomaly
LRU	Longest unused	$O(1)$ w/ map+DLL	Temporal locality	Scan pollution; infeasible in HW
LFU	Fewest accesses	Buckets $O(1)$	Skewed popularity	Stale frequency (needs decay)
Clock	ref-bit=0 on sweep	~0 hit cost	Practical LRU-approx	1-bit coarse → FIFO when all hot

Kernel reality: hardware accessed+dirty bits → clock-family reclaim (Linux active/inactive or MGLRU; Postgres clock-sweep; Redis sampled LRU / 8-bit decayed LFU; Caffeine W-TinyLFU). **Stack algorithms** (LRU, OPT): $\text{resident}(k) \subseteq \text{resident}(k+1)$ → no Belady's anomaly. FIFO: anomaly possible (canonical string: 9 faults @3 frames, 10 @4). **Dirty pages** cost a writeback on eviction → prefer clean victims; background flushers exist for this. **Thrashing:** working set > RAM; fix demand or supply, not swap.

Top Interview Questions

1. Design an LRU cache, $O(1)$ get/put (write it cold). Then thread-safe. Then sharded.
2. Simulate FIFO/LRU/OPT on a reference string, count faults.
3. Why can't the OS do true LRU? What does it do instead? (accessed bit → Clock family)
4. LRU vs LFU — workload-based decision + decay requirement.
5. Belady's anomaly: demonstrate; why is LRU immune?
6. How do Redis / InnoDB / Postgres actually evict? (sampling, midpoint LRU, clock-sweep)
7. What is thrashing and how do you detect/fix it?

Common Mistakes (module-wide)

- LRU-code bugs (map/list desync, no move-to-front on update).
- LFU without decay; FIFO simulation updating on hits.
- "Kernel keeps an exact LRU list per access."
- Claiming Belady's anomaly for LRU; conflating Belady's anomaly with Belady's algorithm.
- Ignoring dirty-page writeback cost and scan pollution.

Mock Interview (self-test, ~20 min)

1. (Code) LRU cache $O(1)$; interviewer adds: "now TTL per key, and eviction must not block gets" — evolve the design.
2. (Simulate) 2 3 2 1 5 2 4 5 3 2 5 2 with 3 frames: fault counts for FIFO, LRU, OPT.
3. (Design) Pick and justify the eviction policy for: (a) CDN edge with 65% one-hit-wonders, (b) DB buffer pool subject to nightly full scans, (c) session cache with strong recency.
4. (Prod) Host: CPU 15%, load 30, disk util 100%, majflt/s 8000. Diagnose and give the fix ladder.
5. (Trap) "We upgraded the cache from 8 GB to 16 GB and the hit rate fell." Give two mechanisms that make this possible (FIFO-family anomaly; sharding/hash changes) and how to verify.